

Task 3

DATABASE DESIGN AND NORMALIZATION IN A COLLEGE CLUB MANAGEMENT

Scenario Overview

The college currently stores all student club information within a single table that combines student details, club details, mentor information, and membership dates. While this approach may appear convenient for small datasets, it introduces structural weaknesses that compromise data integrity, consistency, and scalability. Database normalization was introduced to reduce redundancy and improve data integrity in relational databases ([Codd, 1970](#)).

A relational database should minimize redundancy and prevent anomalies. The given table violates fundamental normalization principles and therefore requires systematic restructuring using formal normalization techniques up to Third Normal Form (3NF).

4.2 IDENTIFYING DATA PROBLEMS

Table 1: Original Unnormalized Club Membership Data

StudentID	StudentName	Email	ClubName	ClubRoom	ClubMentor	JoinDate
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	1/10/2024
2	Bikash	bikash@email.com	Sports Club	R202	Ms. Sita	1/12/2024
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	1/15/2024
3	Nisha	nisha@email.com	Music Club	R101	Mr. Raman	1/20/2024
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	1/18/2024
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	1/22/2024

2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	1/25/2024
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	1/27/2024
3	Nisha	nisha@email.com	Coding Club	Lab1	Mr. Anil	1/28/2024
7	Aman	aman@email.com	Coding Club	Lab1	Mr. Anil	1/30/2024

4.2.1 Data Problems in the Current Table

The current single-table design introduces several structural problems:

1. Data Redundancy

Student information is repeated multiple times.

Example:

Table 2: Example of Data Redundancy in the Original Club Membership Table

StudentID	StudentName	Email	ClubName
1	Asha	asha@email.com	Music Club
1	Asha	asha@email.com	Sports Club

Asha's name and email are duplicated for every club she joins.

This wastes storage and increases maintenance complexity.

- Update Anomaly:** If Asha changes her email address, it must be updated in multiple rows. If one row is missed, inconsistent data appears in the database. This threatens data integrity.
- Insertion Anomaly:** If a new club is created but no student has joined yet, the club cannot be inserted without creating a fake student entry. The structure forces unrelated data to exist together.
- Deletion Anomaly:** If the last student leaves the Coding Club and that row is deleted, all information about the Coding Club is lost. This is dangerous because removing membership removes club data.

4.2.2 Why the Table is Not Normalized

The table violates normalization principles because:

- It mixes multiple entities in one table
- It contains repeated data
- It contains partial and transitive dependencies
- Non-key attributes depend on other non-key attributes

This structure fails to separate logically distinct concepts such as Students, Clubs, and Membership relationships.

4.2.3 Key Definitions

Redundant Data

Data unnecessarily repeated across rows.

Duplicate Data

Identical records stored more than once.

Insert Anomaly

Inability to insert data without unrelated data.

Update Anomaly

Need to modify the same data in multiple rows.

Deletion Anomaly

Loss of unintended data when deleting a record.

4.3 NORMALIZING THE DATA

Database normalization organizes data into structured tables to eliminate redundancy and maintain data integrity ([Elmasri & Navathe, 2016](#)). Normalization is a structured process used to eliminate redundancy and improve data integrity.

4.3.1 First Normal Form (1NF)

Each cell already has a single value. We define a composite primary key of **(StudentID, ClubName)** to uniquely identify each row. This is the 1NF table:

Table 3: Club Membership Data Structured in First Normal Form (1NF)

StudentID (PK)	Student Name	Email	ClubName (PK)	ClubRoom	ClubMentor	JoinDate
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	01/10/2024
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	01/12/2024
2	Bikash	bikash@email.com	Music Club	R101	Mr. Raman	01/22/2024
2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	01/25/2024
3	Nisha	nisha@email.com	Sports Club	R202	Ms. Sita	01/15/2024
3	Nisha	nisha@email.com	Coding Club	Lab1	Mr. Anil	01/27/2024
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	01/20/2024
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	01/18/2024
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	01/30/2024
7	Aman	aman@email.com	Coding Club	Lab1	Mr. Anil	01/28/2024

4.3.2 Second Normal Form (2NF)

Remove partial dependencies by splitting into three tables:

Student Table

Table 4: Student Table After Normalization (2NF)

StudentID (PK)	StudentName	Email
1	Asha	asha@email.com
2	Bikash	bikash@email.com
3	Nisha	nisha@email.com
4	Rohan	rohan@email.com
5	Suman	suman@email.com
6	Pooja	pooja@email.com
7	Aman	aman@email.com

Club Table

Table 5: Club Table After Normalization (2NF)

ClubID (PK)	ClubName	ClubRoom	ClubMentor
C1	Music Club	R101	Mr. Raman
C2	Sports Club	R202	Ms. Sita
C3	Drama Club	R303	Mr. Kiran
C4	Coding Club	Lab1	Mr. Anil

Membership Table

Table 6: Membership Table After Normalization (2NF)

MembershipID (PK)	StudentID (FK)	ClubName (FK)	JoinDate
1	1	Music Club	2024-01-10
2	1	Sports Club	2024-01-15
3	2	Sports Club	2024-01-12
4	2	Drama Club	2024-01-25
5	3	Music Club	2024-01-20
6	3	Coding Club	2024-01-28
7	4	Drama Club	2024-01-18
8	5	Music Club	2024-01-22
9	6	Sports Club	2024-01-27
10	7	Coding Club	2024-01-30

4.3.3 Third Normal Form (3NF)

We introduce a surrogate **ClubID** as the Club table's primary key. This removes transitive dependencies and makes the schema more robust. All non-key attributes now depend only on their table's primary key.

Club Table (3NF):

Table 7: Club Table in Third Normal Form (3NF)

ClubID (PK)	ClubName	ClubRoom	ClubMentor
C01	Music Club	R101	Mr. Raman
C02	Sports Club	R202	Ms. Sita

C03	Drama Club	R303	Mr. Kiran
C04	Coding Club	Lab1	Mr. Anil

Membership Table (3NF – with ClubID):

Table 8: Membership Table in Third Normal Form (3NF)

MembershipID (PK)	StudentID (FK)	ClubID (FK)	JoinDate
1	1	C01	01/10/2024
2	1	C02	01/12/2024
3	2	C01	01/22/2024
4	2	C03	01/25/2024
5	3	C02	01/15/2024
6	3	C04	01/27/2024
7	4	C03	01/20/2024
8	5	C01	01/18/2024
9	6	C02	01/30/2024
10	7	C04	01/28/2024

4.4 ER DIAGRAM

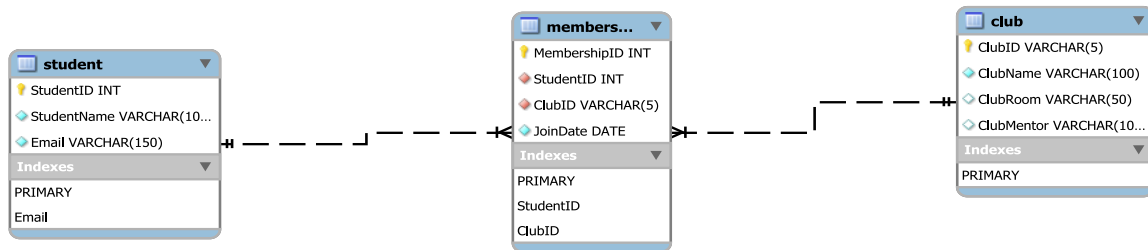


Figure 1: ER diagram of the normalized club membership database.

The ER diagram illustrates a many-to-many relationship between the Student and Club entities. A student may join multiple clubs, while each club can have multiple student members. Entity Relationship diagrams are commonly used to visually represent database entities and their relationships ([Connolly & Begg, 2015](#)). This relationship is implemented through the Membership table, which acts as a junction table connecting the two entities through foreign keys.

The relationship between entities is:

- One Student can join many Clubs
- One Club can have many Students

This is a many-to-many relationship resolved through the Membership entity.

4.5 Basic SQL Operations

Table Creation Setup:

SQL is the standard language used to manage and retrieve data in relational database systems ([Silberschatz et al., 2019](#)). The following SQL statements demonstrate the implementation of the normalized database schema and basic database operations.


```

40 • CREATE TABLE Student (
41     StudentID INT PRIMARY KEY,
42     StudentName VARCHAR(100) NOT NULL,
43     Email VARCHAR(150) NOT NULL UNIQUE
44 );
45
46 • CREATE TABLE Club (
47     ClubID VARCHAR(5) PRIMARY KEY,
48     ClubName VARCHAR(100) NOT NULL,
49     ClubRoom VARCHAR(50),
50     ClubMentor VARCHAR(100)
51 );
52
53 • CREATE TABLE Membership (
54     MembershipID INT PRIMARY KEY AUTO_INCREMENT,
55     StudentID INT NOT NULL,
56     ClubID VARCHAR(5) NOT NULL,
57     JoinDate DATE NOT NULL,
58     FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
59     FOREIGN KEY (ClubID) REFERENCES Club(ClubID)
60 );

```

Figure 2: Table Creation for Student, Club, and Membership Entities

1. Insert a New Student

```

64 -- Step 6: Add NEW student
65
66 • INSERT INTO Student (StudentName, Email)
67 VALUES ('Priya', 'priya@email.com');
68

```

Figure 3: SQL Query for Inserting a New Student Record

Adds student Priya with StudentID 8 and her email into the student table.

2. Insert a New Club

```
69 • INSERT INTO Club (ClubID, ClubName, ClubRoom, ClubMentor)
70   VALUES ('C05', 'Art Club', 'R404', 'Ms. Deepa');
71
```

Output :

Action Output

	#	Time	Action	Message
✓	26	21:30:43	SELECT * FROM Student LIMIT 0, 1000	8 row(s) returned
✓	27	21:31:42	SELECT * FROM Club LIMIT 0, 1000	4 row(s) returned
✓	28	22:57:36	INSERT INTO Club (ClubID, ClubName, ClubRoo...	1 row(s) affected

Figure 4: SQL Query for Adding a New Club into the Club Table

Add a new Art Club in room R404, mentored by Ms. Deepa.

3. Display All the Student

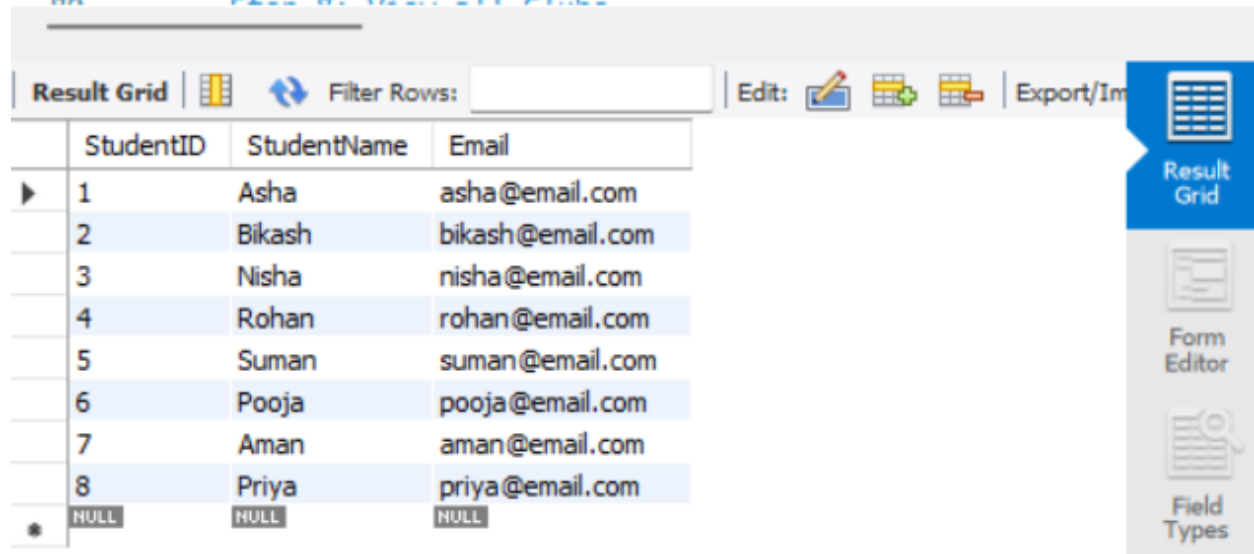
76 -- Step 7: View all Students

77

78 • `SELECT * FROM Student;`

79

80 -- Step 8: View all Clubs



	StudentID	StudentName	Email
▶	1	Asha	asha@email.com
	2	Bikash	bikash@email.com
	3	Nisha	nisha@email.com
	4	Rohan	rohan@email.com
	5	Suman	suman@email.com
	6	Pooja	pooja@email.com
	7	Aman	aman@email.com
	8	Priya	priya@email.com
*	NULL	NULL	NULL

Figure 5: SQL Query Result Displaying All Records in the Student Table

Retrieves all rows and columns from the Student table.

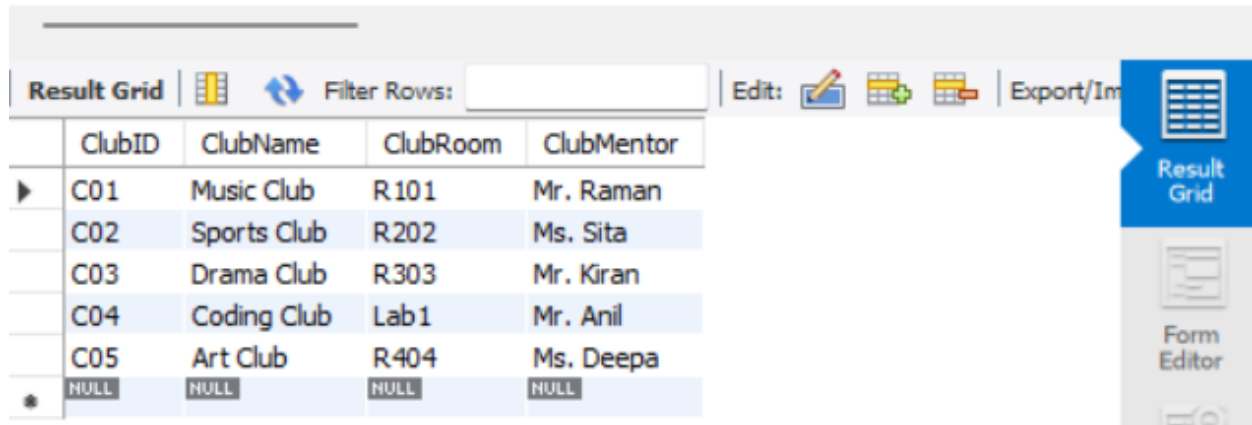
4. Display All Clubs

86

87 • `SELECT * FROM Club;`

88

89 `-- Step 9: View all Memberships with JOIN`



	ClubID	ClubName	ClubRoom	ClubMentor
▶	C01	Music Club	R101	Mr. Raman
	C02	Sports Club	R202	Ms. Sita
	C03	Drama Club	R303	Mr. Kiran
	C04	Coding Club	Lab1	Mr. Anil
	C05	Art Club	R404	Ms. Deepa
*	NULL	NULL	NULL	NULL

Figure 6: SQL Query Result Displaying All Records in the Club Table

Retrieves all rows and columns from the Club table.

4.6 SQL JOIN OPERATION

```
91 • SELECT
92     s.StudentID,
93     s.StudentName,
94     s.Email,
95     c.ClubName,
96     c.ClubRoom,
97     c.ClubMentor,
98     m.JoinDate
99 FROM
100     Membership m
101     JOIN Student s ON m.StudentID = s.StudentID
102     JOIN Club c ON m.ClubID = c.ClubID
103 ORDER BY
104     s.StudentID, m.JoinDate;
```

Figure 7: SQL JOIN Query Combining Student, Club, and Membership Tables

Explanation:

- FROM Membership m — starts from the junction table.

- JOIN Student s ON m.StudentID = s.StudentID — links each membership to the matching student.
- JOIN Club c ON m.ClubID = c.ClubID — links each membership to the matching club.
- ORDER BY — sorts result alphabetically by student name, then by date.

Result:

Result Grid							
Filter Rows:			Export:		Wrap Cell Content:		
	StudentID	StudentName	Email	ClubName	ClubRoom	ClubMentor	JoinDate
▶	1	Asha	asha@email.com	Music Club	R101	Mr. Raman	2024-01-10
	1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	2024-01-12
	2	Bikash	bikash@email.com	Music Club	R101	Mr. Raman	2024-01-22
	2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	2024-01-25
	3	Nisha	nisha@email.com	Sports Club	R202	Ms. Sita	2024-01-15
	3	Nisha	nisha@email.com	Coding Club	Lab 1	Mr. Anil	2024-01-27
	4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	2024-01-20
	5	Suman	suman@email.com	Music Club	R101	Mr. Raman	2024-01-18
	6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	2024-01-30
	7	Aman	aman@email.com	Coding Club	Lab 1	Mr. Anil	2024-01-28
	8	Priya	priya@email.com	Music Club	R101	Mr. Raman	2024-02-01

Figure 8: Result of JOIN Operation Displaying Student Club Membership Details

4.7 Reflection

1. How Normalization Reduces Data Repetition

Before normalization, the original flat table stored the same fact multiple times. The detail that Sports Club is in room R202 under Ms. Sita appeared in three separate rows — once for every student who joined. With 100 students in Sports Club, that single fact would be stored 100 times.

Normalization eliminates this by storing each fact exactly once. After normalization, Sports Club's room and mentor are stored in a single row of the Club table. All membership records simply reference ClubID C02, without repeating any club details. This dramatically reduces storage and ensures data is never unnecessarily duplicated. The underlying principle is the "single source of truth" every fact lives in one place only, and all other tables point to it rather than copying it.

2. How Normalization Improves Data Accuracy

When the same fact is stored in many rows, there is a high risk that an update to one row is not applied to all others. For example, if Sports Club's room is updated to R205 in two rows but left as R202 in a third, the database becomes contradictory — it gives different answers to the same question. There is no way to know which answer is correct.

Normalization ensures there is only one row to update. Changing Sports Club's room in the Club table immediately propagates that change to every query that references it. The database is always consistent. Furthermore, foreign key constraints enforce referential integrity — a membership cannot reference a ClubID that does not exist in the Club table. This prevents orphaned records and ensures all relationships are always valid, making the database a trustworthy system.

3. Why JOIN Operations Are Necessary After Normalization

Normalization deliberately distributes data across multiple tables. While this keeps the data clean and non-redundant, it means that information which was once in a single row is now spread across Student, Club, and Membership tables. When a user needs a combined view such as "which student joined which club on which date" that information must be reassembled from the separate tables. This is precisely what JOIN operations do.

A JOIN temporarily combines rows from two or more tables based on a matching key column (such as StudentID or ClubID), returning a unified result set. Without JOINS, a developer would have to query each table separately and manually match results in application code an approach that is slow, error-prone, and unscalable.

JOINS are therefore the necessary complement to normalization: normalization separates data to eliminate redundancy, while JOINS reunite the data at query time to make it usable. The two techniques work together store data cleanly once, query it powerfully anywhere. In the college club system, a JOIN across Student, Membership, and Club is the only way to produce the human-readable report showing a student's name, club, and join date in a single result.

4.8 CRITICAL EVALUATION OF DATABASE DESIGN

The normalized structure significantly improves:

Data Integrity

Redundancy is eliminated.

Each entity is stored once.

Scalability

New students and clubs can be added independently.

Consistency

Updates occur in a single location.

Query Efficiency

JOIN operations allow relational retrieval without duplication.

However, normalization introduces:

- More tables
- Need for JOIN queries
- Slightly increased query complexity

CONCLUSION

This report explored several fundamental concepts in computer science that influence how modern digital systems represent, process, transmit, and manage data. The analysis demonstrated how theoretical principles are directly connected to practical computing problems.

The first section examined the role of encoding formats in secure data exchange across modern network protocols. Formats such as Base64, hexadecimal, ASCII, and URL encoding were evaluated to understand their contribution to data representation and interoperability. The discussion showed that while encoding ensures compatibility between systems and prevents data misinterpretation, it does not provide security by itself. Secure communication instead depends on the integration of encoding with encryption protocols such as TLS and additional mechanisms including authentication and validation.

The second section analysed the classroom seating arrangement scenario through the perspective of computational complexity theory. By applying the concepts of P and NP problems, it became clear that verifying a seating arrangement can be performed efficiently, whereas finding a valid arrangement becomes increasingly difficult due to factorial growth in possible permutations. The comparison between brute force and heuristic strategies illustrated how practical problem-solving often requires balancing optimality with computational efficiency. The final section addressed database design through the normalization of a college club membership system. By transforming the original unstructured table into first, second, and third normal forms, the design eliminated redundancy and minimized data anomalies. The development of an ER diagram and SQL implementation demonstrated how relational database principles improve data consistency and maintainability.

Overall, this report highlights the importance of foundational computer science concepts in designing efficient, secure, and scalable information systems.